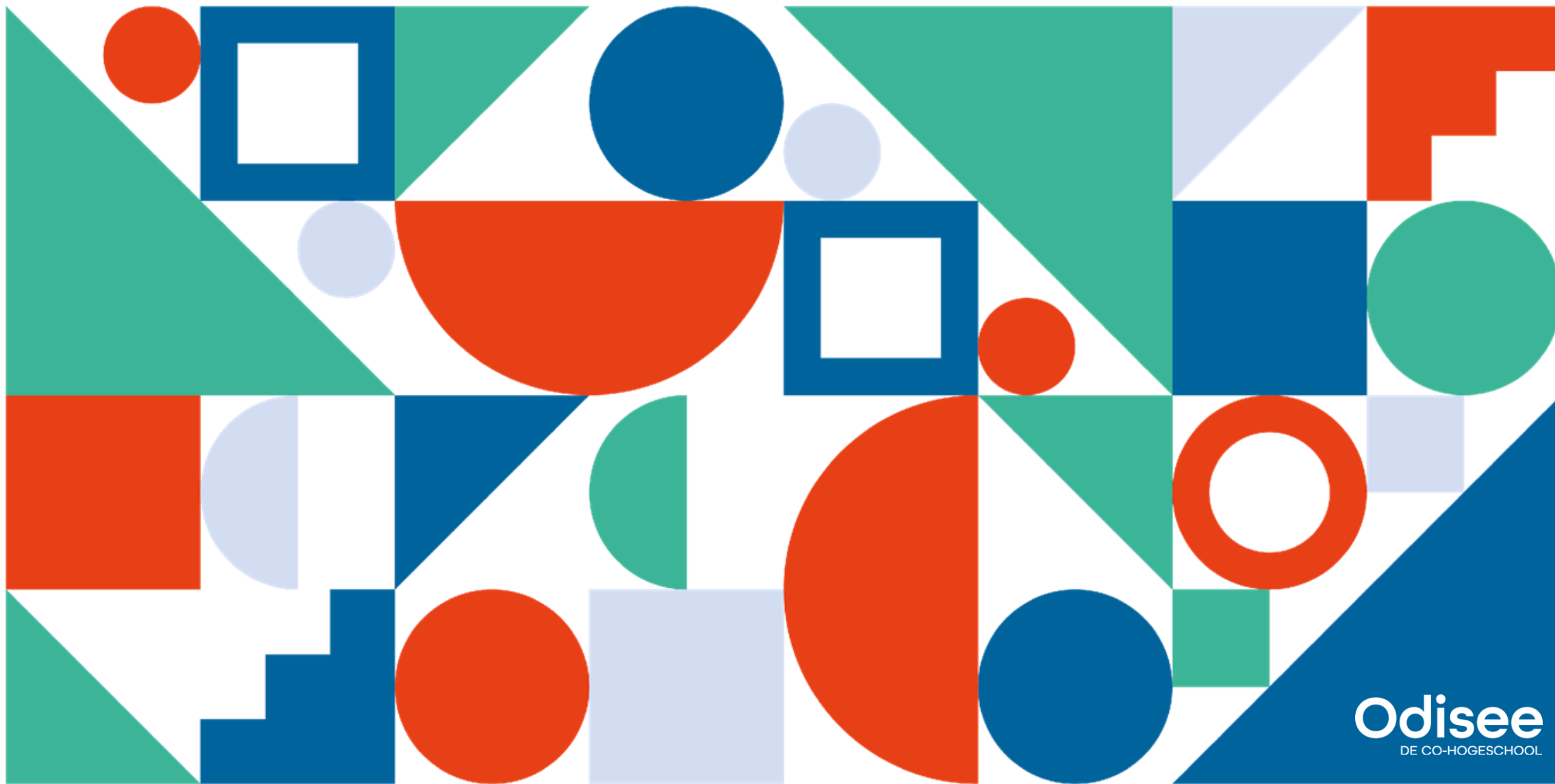




Odisee
DE CO-HOGESCHOOL



1.

Scalaire Subqueries

```
SELECT *  
FROM employees  
WHERE salary < (  
    SELECT AVG(salary)  
    FROM employees);
```

Vertaal deze query naar het Nederlands.

```
SELECT *  
FROM employees  
WHERE salary < (  
    SELECT AVG(salary)  
    FROM employees);
```

Geef de employees
waarvoor geldt dat hun salaris kleiner is dan
het gemiddelde salaris (v/d employees)

Tussenstappen

Uitvoeren van de query in 2 stappen:

SELECT *

FROM *employees*

WHERE *salary* < (

SELECT **AVG**(*salary*)
FROM *employees*);

2

1

Resultaat uitvoeren van subquery 1 :

AVG(SALARY)

8775

Tussenstappen

Substitutie van resultaat subquery 1 in hoofdquery 2:

```
SELECT *  
FROM employees  
WHERE salary < 8775;
```

Resultaat uitvoeren van hoofdquery 2 :

employee_id	first_name	last_name	salary
104	Bruce	Ernst	6000
107	Diana	Lorentz	4200
124	Kevin	Mourgos	5800
...	...	...	...

Kunnen we dit zonder subquery?

Het moet en zal lukken zonder subquery, poging 1:

```
SELECT *  
FROM employees  
WHERE salary < AVG(salary);
```

Waarom “Error Code: 1111. Invalid use of group function”?

❌ 67 17:22:18 SELECT * FROM employees WHERE salary < AVG(salary) L... Error Code: 1111. Invalid use of group function

Kunnen we dit zonder subquery?

Het moet en zal lukken zonder subquery, poging 1:

```
SELECT *  
FROM employees  
WHERE salary < AVG(salary);
```

Waarom “Error Code: 1111. Invalid use of group function”?

Een group function zoals AVG moet in de HAVING clause staan! Omdat WHERE wordt uitgevoerd voor de groepfuncties worden uitgevoerd.

Kunnen we dit zonder subquery?

Het moet en zal lukken zonder subquery, poging 2:

```
SELECT *  
FROM employees  
HAVING salary < AVG(salary);
```

Waarom “Error Code: 1463. Non-grouping field 'salary' is used in HAVING clause”?

69 17:26:52 SELECT * FROM employees HAVING salary < AVG(...) Error Code: 1463. Non-grouping field 'salary' is used in HAVING clause

Kunnen we dit zonder subquery?

Het moet en zal lukken zonder subquery, poging 2:

```
SELECT *  
FROM employees  
HAVING salary < AVG(salary);
```

Waarom “Error Code: 1463. Non-grouping field 'salary' is used in HAVING clause”?

HAVING gebeurt na GROUP BY. De kolom salary bestaat niet meer nadat de GROUP BY is uitgevoerd. Enkel groepsfuncties en expressies welke in de GROUP BY staan blijven over.

Volgorde van de bewerkingen

Met subquery

- FROM employees
- GROUP BY <niets>
- SELECT AVG(salary)
- <substitutie>
- FROM employees
- WHERE salary < 8775
- SELECT *

Poging zonder subquery

- FROM employees
- WHERE salary < AVG(salary)
- GROUP BY <niets>
- SELECT *

Probleem: WHERE wordt voor de GROUP BY uitgevoerd maar wij willen de AVG functie uitvoeren na de GROUP BY.

Welke job heeft het laagste minimum salaris (min_salary)?

Welke zijn de 2 stappen/queries om dit te berekenen?

Welke job heeft het laagste minimum salaris (min_salary)?

1) Wat is het laagste min_salary?

```
SELECT      MIN(min_salary)
FROM        jobs;
```

2) Welke job heeft als min_salary 2000?

```
SELECT job_id, job_title
FROM jobs
```

```
WHERE min_salary = 2000;
```

Voorbeeld substitutie: 2 Queries => 1 Query

```
SELECT      job_id, job_title
FROM        jobs
WHERE min_salary = (
            SELECT      MIN(min_salary)
            FROM        jobs);
```

- A. Gemakkelijk. Welke employee heeft het hoogste loon?
Welke zijn de 2 stappen/queries om dit te berekenen?**

- B. Wie werkt er in de afdeling waar er 5 mensen werken?**

- C. Uitdaging. Welke afdeling heeft een gemiddeld salaris dat groter is dan het salaris van de manager van de afdeling marketing?**

2. Subqueries met meerdere rijen

Probleemstelling

Wie werkt er in de afdeling waar er 5 mensen werken?

SELECT * **FROM** employees

WHERE department_id = (

SELECT department_id **FROM** employees

GROUP BY department_id

HAVING COUNT(employee_id) = 5);

COUNT(employee_id) < 5



Wie werkt er in de afdeling waar er **minder dan 5** mensen werken?

Probleemstelling

Error Code: 1242. Subquery returns more than 1 row

De subquery vindt meerdere afdelingen!

```
SELECT department_id FROM employees  
GROUP BY department_id  
HAVING COUNT(employee_id) < 5;
```

department_id	aantal
NULL	1
10	1
20	2
60	3
80	3
90	3
110	2

Probleemstelling

De operator = vergelijkt 2 waarden met elkaar.

Waarde links: het department_id van de rij.

Waarde rechts: het resultaat van de subquery welke meer dan 1 waarde bevat, een opsomming van meerdere department_ids.

SELECT * FROM employees

WHERE department_id **=** (

SELECT department_id **FROM** employees

GROUP BY department_id

HAVING COUNT(employee_id) < 5);

Probleemstelling

De operator IN kan wel meerdere waarden aan

```
SELECT * FROM employees
WHERE department_id IN (
    SELECT department_id FROM employees
    GROUP BY department_id
    HAVING COUNT(employee_id) < 5 );
```

D. In de `job_history` tabel wordt een nieuwe rij toegevoegd elke keer een employee verandert van job of departement. Wie nooit veranderde, staat er dus niet in.

D1. Toon de id en namen van de employees die **NOOIT** veranderden van job of departement.

D2. Toon de id en namen van de employees die **OOIT** veranderden van job of departement.

D3. Uitdaging. Toon de id en namen van de employees die ooit veranderden van departement.



3.

Multi row, multi column subqueries

Wat geeft een subquery terug?

Wat geeft een scalaire subquery terug?

```
SELECT * FROM employees  
WHERE salary < (  
    SELECT AVG(salary) FROM employees);
```

Wat geeft een subquery terug?

Wat geeft een scalaire subquery terug?

1 Waarde

Wat geeft een single column subquery terug?

```
SELECT * FROM employees
```

```
WHERE department_id IN (
```

```
    SELECT department_id FROM departments
```

```
    WHERE location_id = 1700);
```

Wat geeft een subquery terug?

- | | |
|---|----------|
| Wat geeft een scalaire subquery terug? | 1 Waarde |
| Wat geeft een single column subquery terug? | 1 kolom |
| Wat geeft een subqueries terug die met meerdere kolommen en rijen teruggeeft? | |

```
SELECT * FROM employees;
```

Wat geeft een subquery terug?

Wat geeft een scalaire subquery terug?	1 Waarde
Wat geeft een single column subquery terug?	1 kolom
Wat geeft een subqueries terug die met meerdere kolommen en rijen teruggeeft?	1 tabel!

```
SELECT * FROM employees;
```

We kunnen een subquery die een tabel teruggeeft in een query gebruiken waar er een tabel gebruikt wordt.

Vraag: Welke medewerkers hebben een maandsalaris hoger dan het gemiddelde van hun eigen afdeling?

Wat zijn de stappen waarin het dit opsplitsen?

Stap 1: Bereken het gemiddelde salaris per afdeling.

Stap 2: Geef de werknemers welke meer verdienen dan het gemiddelde (van hun afdeling) uit stap 1.

Voorbeeld

Stap 1: Bereken het gemiddelde salaris (van de werknemers) per afdeling.

```
SELECT      department_id , AVG(salary) AS gemiddelde
FROM        employees
GROUP BY    department_id ;
```

Stap 2: Geef de werknemers welke meer verdienen dan het gemiddelde (van hun afdeling) uit stap 1.

```
SELECT * FROM employees
JOIN stap1 ON stap1.department_id = employees.department_id
WHERE employees.salary > stap1.gemiddelde;
```

Voorbeeld

Substitutie: vervang stap1 door de subquery van stap 1.
Hernoem stap1 naar toepasselijkere naam: gem_salaris_afdeling.

```
SELECT * FROM employees JOIN  
    (SELECT      department_id , AVG(salary) AS gemiddelde  
    FROM          employees  
    GROUP BY      department_id) AS gem_salaris_afdeling  
    ON gem_salaris_afdeling.department_id = employees.department_id  
WHERE employees.salary > gem_salaris_afdeling.gemiddelde;
```

WITH syntax

Splits de opgave op in meerdere stappen en zet elke stap in een subquery.
Kies passende namen voor elke stap.

WITH stap1 **AS**

()

<- subquery stap 1

, stap2 **AS**

()

<- subquery stap 2

SELECT ...

FROM ...

<- stap 3

WITH syntax Voorbeeld

Herschrijf het vorige voorbeeld en plaats de subquery in de WITH.

```
WITH gem_salaris_afdeling AS (  
    SELECT      department_id , AVG(salary) AS gemiddelde  
    FROM        employees  
    GROUP BY    department_id)  
SELECT * FROM employees JOIN gem_salaris_afdeling  
    ON gem_salaris_afdeling.department_id = employees.department_id  
WHERE employees.salary > gem_salaris_afdeling.gemiddelde;
```

D. Splits op in stappen en pas substitutie toe. Schrijf het 1 maal met en 1 maal zonder WITH.

Wat is het grootste aantal departementen op 1 locatie? Vb locatie 1700 heeft 4 departementen dus het grootste aantal is 4.

E. Splits op in stappen en pas substitutie toe. Schrijf het 1 maal met en 1 maal zonder WITH.

Wat is het grootste aantal departementen op 1 locatie? Vb locatie 1700 heeft 4 departementen dus het grootste aantal is 4.

Stap 1: bereken het aantal departementen per locatie

Stap 2: Geef het grootste aantal van stap 1.